

Inspiring Excellence

LAB PROJECT REPORT

Mushroom Edibility Classification Using Machine Learning

SUMMER 2026

COURSE CODE: **CSE422**
COURSE TITLE: **ARTIFICIAL INTELLIGENCE**

Table of Contents

Section No.	Content	Page No
1	Introduction	2
2	Dataset Description	2-6
3	Dataset Pre-processing	6-7
4	Data Splitting	7
5	Model Training & Testing (Supervised)	7-8
6	Comparison Analysis	8-11
7	Model Training & Testing (Unsupervised)	11-14
8	Conclusion	14

1. Introduction

Mushroom identification is an important classification problem because misidentifying poisonous species as edible can pose serious health risks. This project investigates applying machine learning to classify mushrooms as edible or poisonous based on observable characteristics such as cap shape, color, gill features, stem dimensions, habitat, and season.

The study uses the Secondary Mushroom Dataset, consisting of 61,069 mushroom instances. The problem is formulated as a binary classification task. Three supervised learning algorithms—K-Nearest Neighbors (KNN), Logistic Regression, and a Neural Network—are trained and evaluated to compare their classification performance. Additionally, K-Means clustering is applied to examine the underlying structure of the dataset and its relationship with mushroom edibility.

The report presents the dataset, preprocessing procedures, model development and evaluation, K-Means analysis, and the key findings of the study.

2. Dataset Description

Number of features: The raw dataset has 20 input features plus 1 target column ("class"), for 21 columns in total. After pre-processing, 5 features with excessive missing data were dropped, leaving 15 usable input features (3 numeric, 12 categorical) before one-hot encoding.

Classification: This is a classification problem, specifically binary classification because the target column "class" has two discrete values: **'**e' (edible) and 'p' (poisonous).

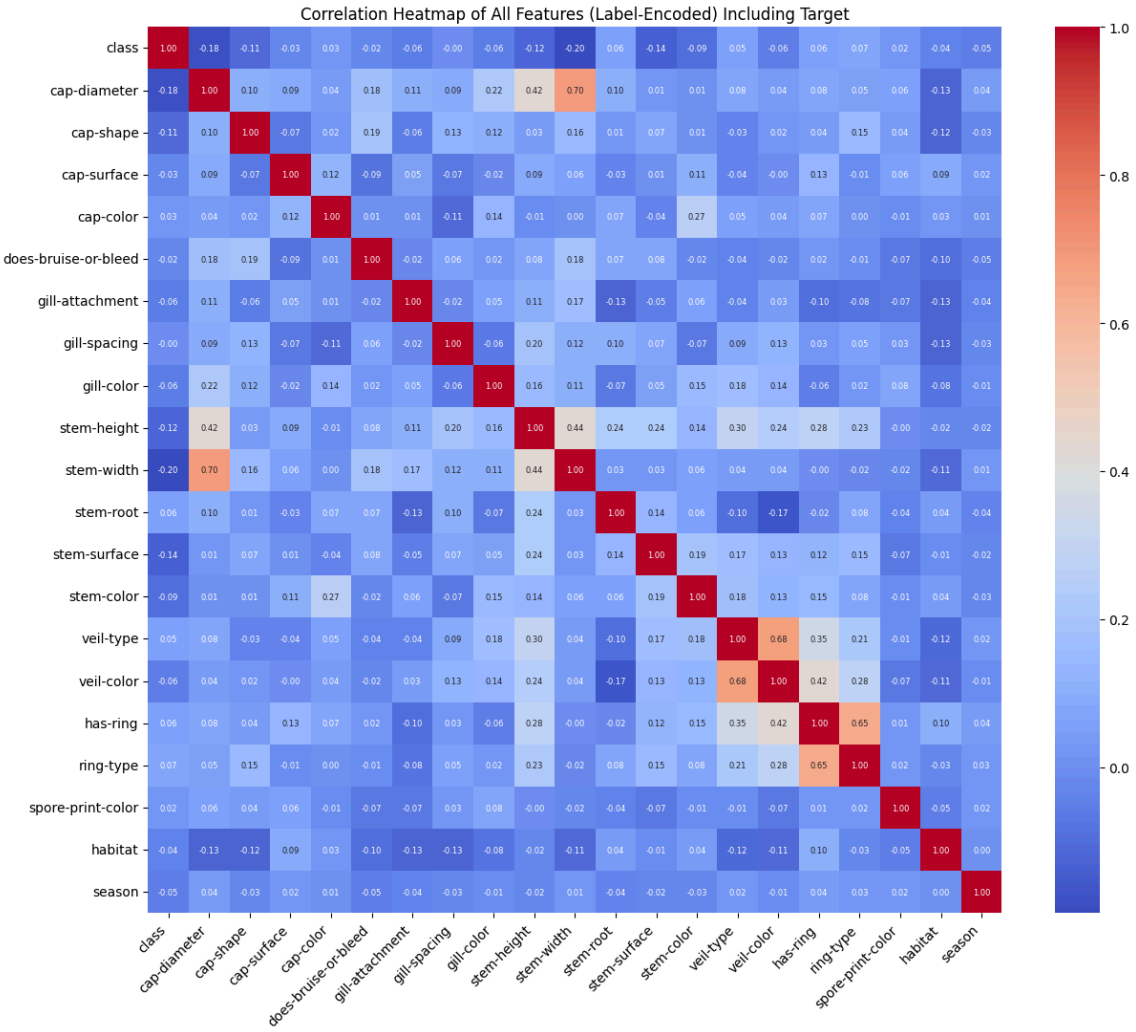
Number of data points: Number of data points: The dataset contains 61,069 rows (instances), representing hypothetical mushrooms from 173 real mushroom species.

Feature types: The dataset is a mix of quantitative and categorical features:

- **Quantitative:** cap-diameter, stem-height, stem-width.
- **Categorical:** cap-shape, cap-surface, cap-color, does-bruise-or-bleed, gill-attachment, gill-spacing, gill-color, stem-root, stem-surface, stem-color, veil-type, veil-color, has-ring, ring-type, spore-print-color, habitat, season.

The categorical variables need encoding: Yes. Since the categorical features are non-ordinal, they are one-hot encoded for modeling to avoid imposing false order. Label encoding is used temporarily only for the correlation heatmap.

Correlation Heatmap:



Key Correlation Statistics from the Heatmap:

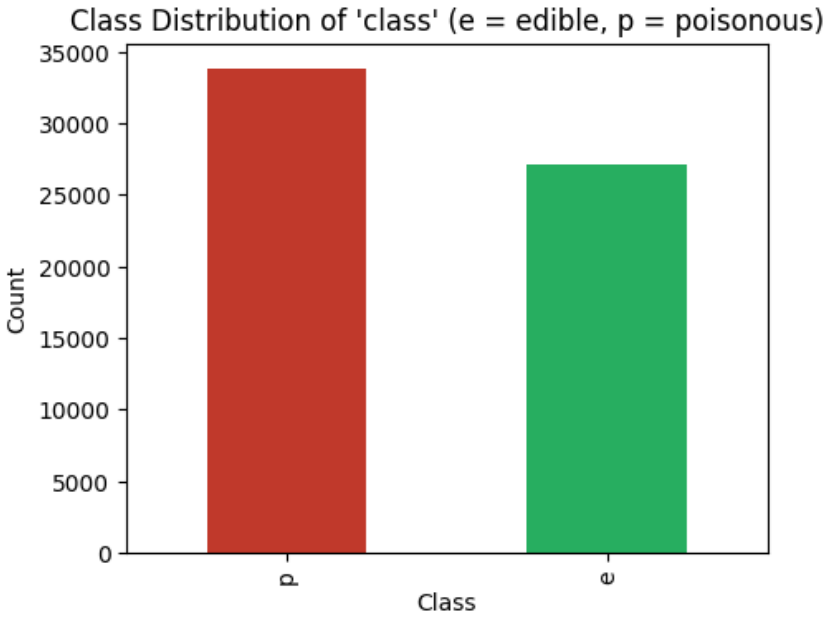
Feature	Correlation with Target
stem-width	-0.20
cap-diameter	-0.18
stem-surface	-0.14

stem-height	-0.12
cap-shape	-0.11
stem-color	-0.10
ring-type	+0.07
stem-root	+0.06

The highest correlation with the target is stem-width ($|r| \approx 0.20$), indicating that no single feature strongly predicts edibility. Instead, edibility is likely determined by combinations of multiple categorical features, which may explain why non-linear models can still achieve strong performance despite low individual correlations.

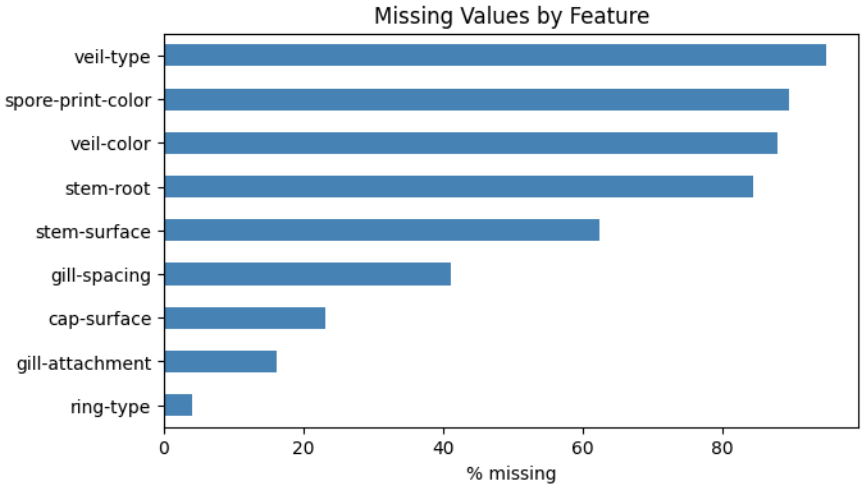
Imbalanced Dataset

For the output feature, the two classes are not perfectly equal but are reasonably close: 33,888 instances (55.5%) are poisonous, and 27,181 instances (44.5%) are edible.

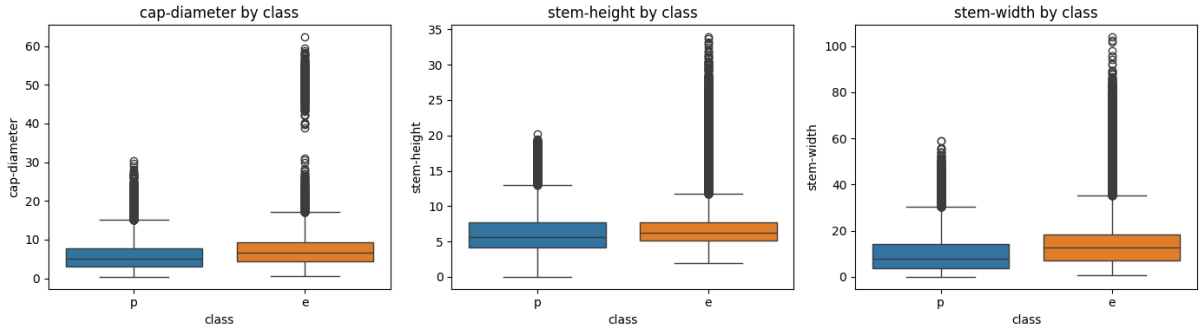


Exploratory Data Analysis

We first checked data completeness across all columns, since several categorical features turned out to have very large numbers of missing entries :



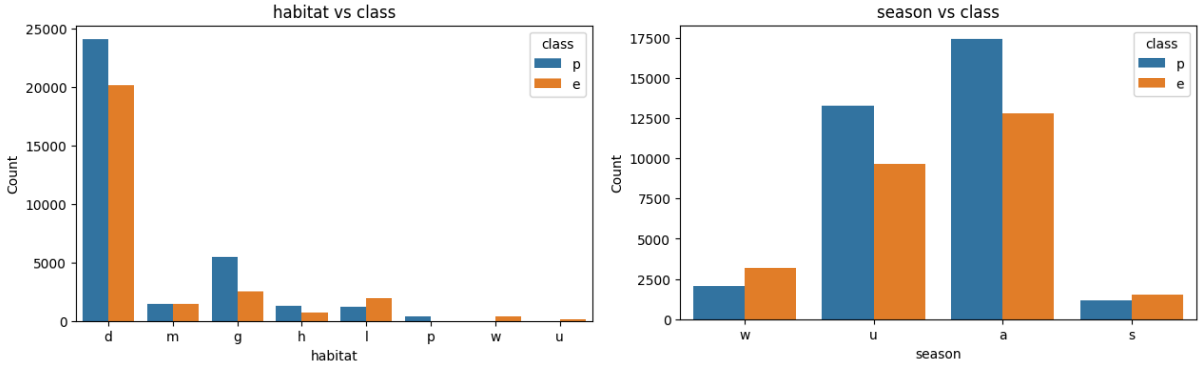
Next, we looked at how the three numeric features are distributed and whether they differ by class:



Insights:

- Poisonous mushrooms are slightly smaller on average, but the distributions overlap substantially.
- Outliers appear in all three numeric features, affecting roughly 3–5% of rows.

Edibility was also examined across the categorical features habitat and season:



Both habitat and season show variation in edibility across categories, indicating that these categorical features provide predictive signal and should be included using one-hot encoding.

3. Dataset Pre-processing

- **Handling Null Values**

  ○ **Fault:** Machine learning models cannot directly process Null values, and categorical encoding cannot be performed while Null values exist.

  ○ **Solution:** Instead of deleting rows or imputing highly missing columns, we handled missing values based on their percentage of missingness.

Feature	% Missing	Action Taken
veil-type	94.797688	Dropped, constant
spore-print-color	89.595376	Dropped
veil-color	87.861272	Dropped
stem-root	84.393064	Dropped
stem-surface	62.427746	Dropped
gill-spacing	41.040462	Imputed as “missing”
cap-surface	23.121387	Imputed as “missing”
gill-attachment	16.184971	Imputed as “missing”
ring-type	4.046243	Imputed as “missing”

Here, Five categorical columns — veil-type, veil-color, spore-print-color, stem-root, and stem-surface — had more than 50% missing values and were dropped. veil-type was also constant and provided no useful information. The remaining four columns — gill-spacing, cap-surface, gill-attachment, and ring-type — were imputed using a new “missing” category, preserving potentially informative missingness.

- **Encoding**

  ○ **Fault:** Machine learning models require numerical input and cannot directly process categorical string values.

○ **Solution:** The target variable class was mapped to numerical values, where 0 = edible and 1 = poisonous. Since the categorical features are nominal and have no natural order, One-Hot Encoding was applied using `get_dummies()`. The first category was dropped for each feature to avoid redundant columns, resulting in 81 binary features from the 12 remaining categorical columns.

- **Feature Scaling**

  ○ **Fault:** Features with larger numerical ranges can have a greater influence on distance- and gradient-based models, which may introduce bias.

  ○ **Solution:** We applied StandardScaler to standardize the features to a mean of approximately **0** and a standard deviation of **1**. Scaling was applied to the training data first and then used to transform the test data to prevent data leakage. Scaling was used for KNN, Logistic Regression, and Neural Network, which are sensitive to feature magnitude.

4. Dataset Splitting

The dataset was split into 80% training (48,855 instances) and 20% testing (12,214 instances) using a stratified split, preserving the ~55/45 class ratio. random_state=42 was used for reproducibility. Model selection used 5-fold cross-validation for performance evaluation on the training set.

5. Model Training & Testing (Supervised)

As we have a classification problem, for supervised model training, we used:

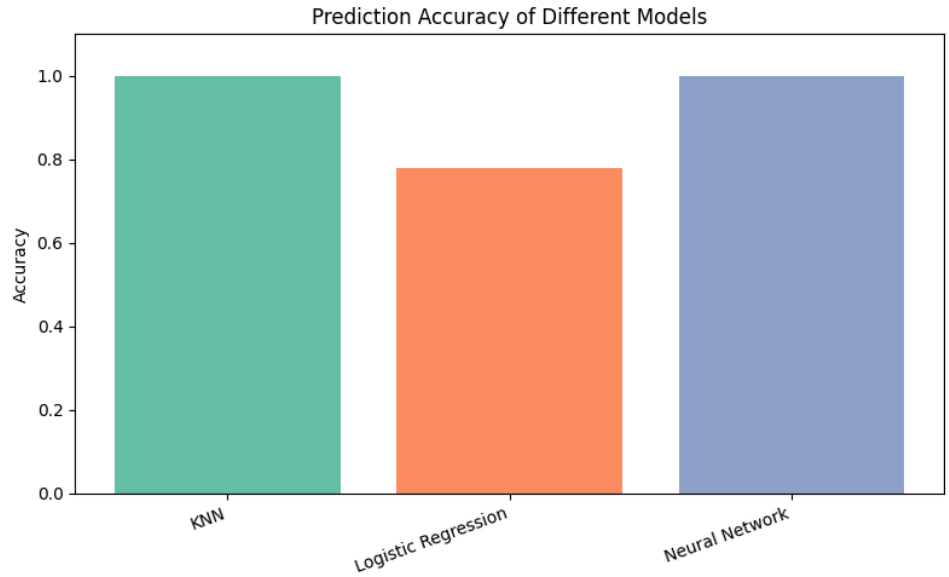
1. K-Nearest Neighbors (KNN)
2. Logistic Regression
3. Neural Network

Model Cross-Validation Results

Model	CV Accuracy
KNN	0.9999 +/- 0.0002
Logistic Regression	0.7800 +/- 0.0041
Neural Network	0.9996 +/- 0.0004

6. Comparison Analysis

- **Prediction Accuracy**



  - KNN: 100%
  - Logistic Regression: 78.01%
  - Neural Network: 99.98%

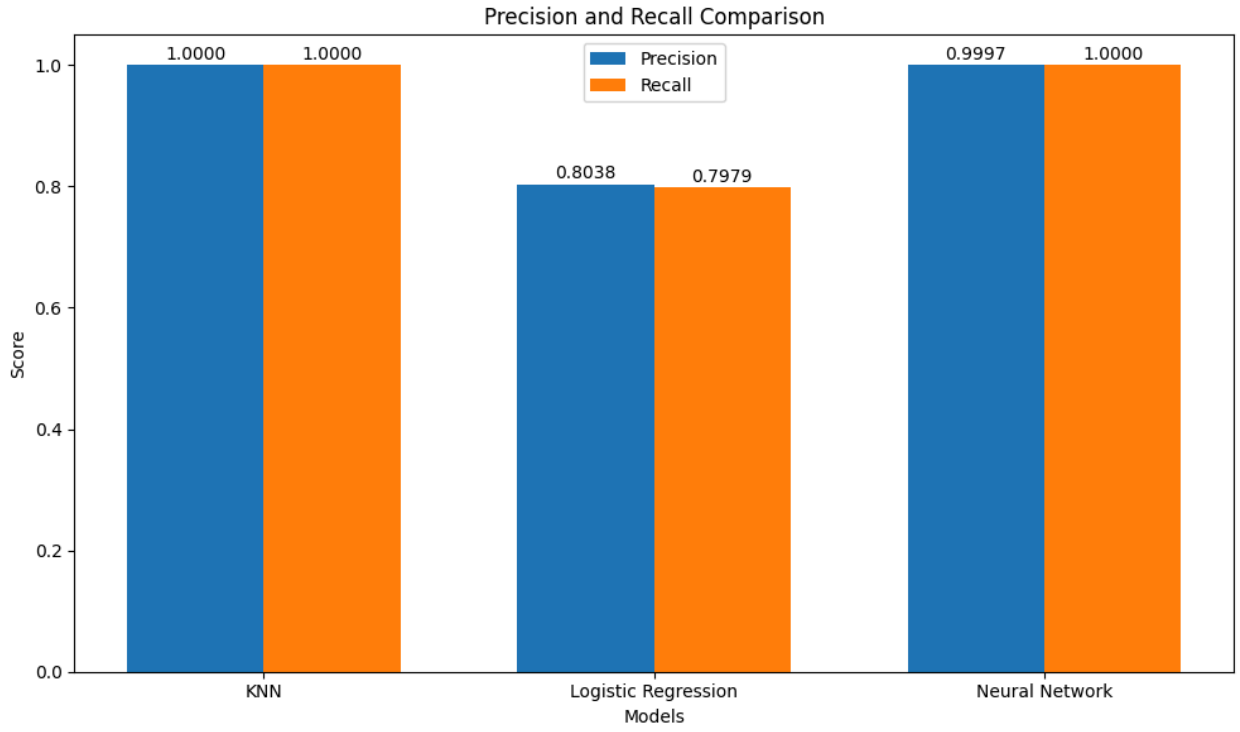
  Here, KNN and Neural Network performed better than Logistic Regression.

- **Precision and Recall Comparison**

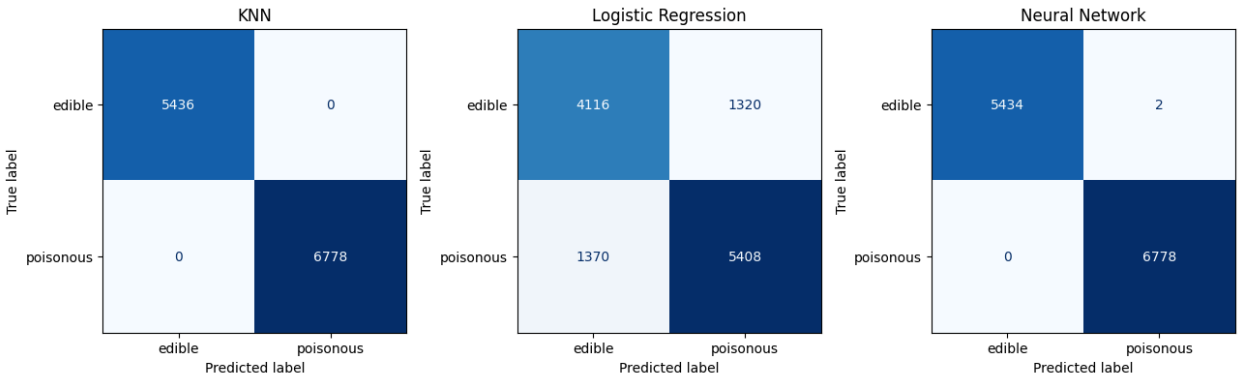
  - KNN
    - Precision: 100.00%
    - Recall: 100.00%

○ Logistic Regression
- Precision: 80.38%
- Recall: 79.79%

○ Neural Network
- Precision: 99.97%
- Recall: 100.00%



- **Confusion Matrices**

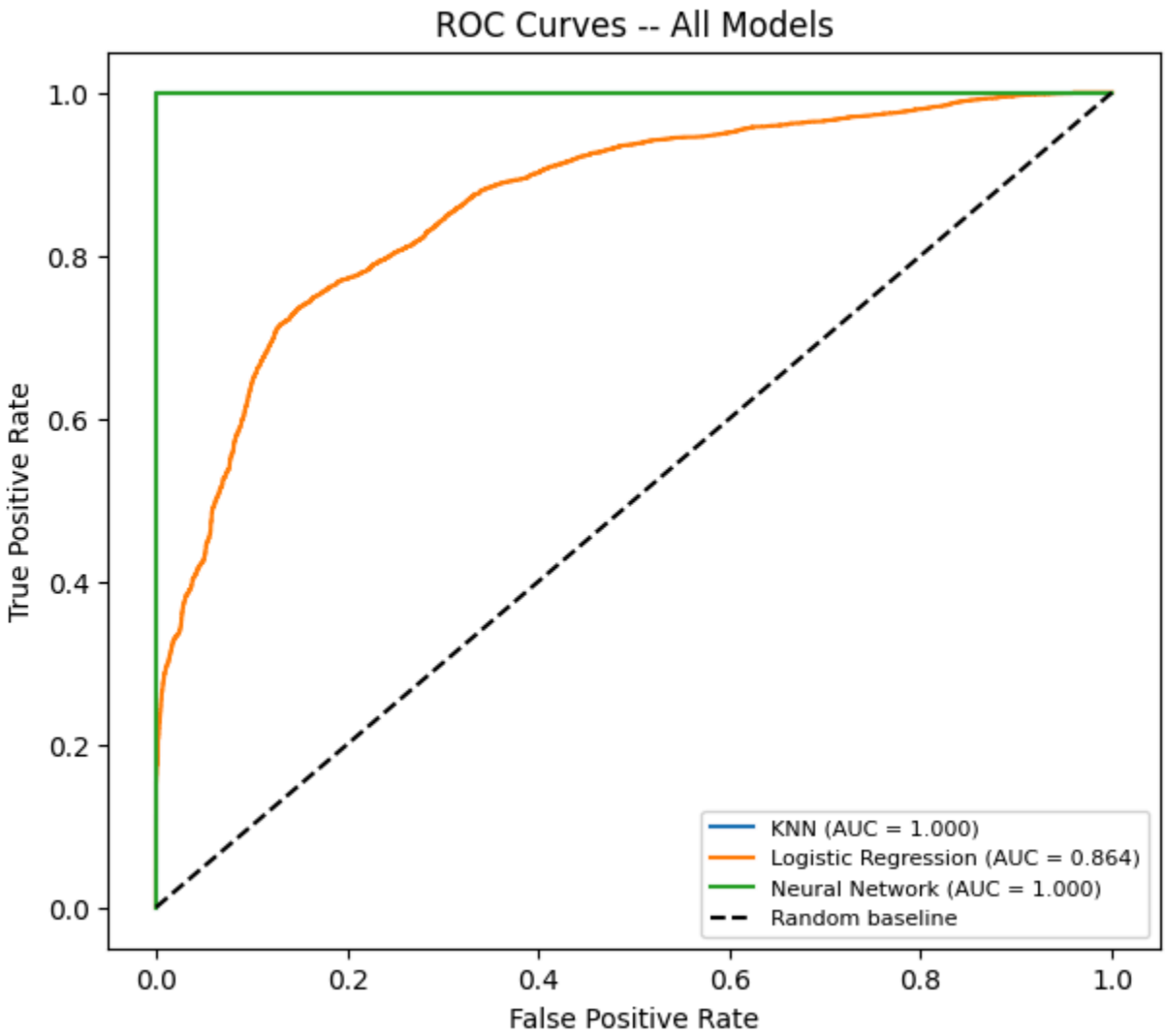


○ KNN: Achieved 100% accuracy with no misclassifications, as all samples were correctly classified into their respective classes.

○ Logistic Regression: More misclassifications compared with KNN, with low and medium classes predicted more accurately than the high class.

○ Neural Network: Very high diagonal values, indicating strong classification performance, with very few misclassifications.

- **AUC Score and ROC Curves**



○ KNN: 1.000

○ Logistic Regression: 0.864

○ Neural Network: 1.000

The models achieved high AUC scores, indicating excellent discriminatory ability and strong performance in distinguishing between classes.

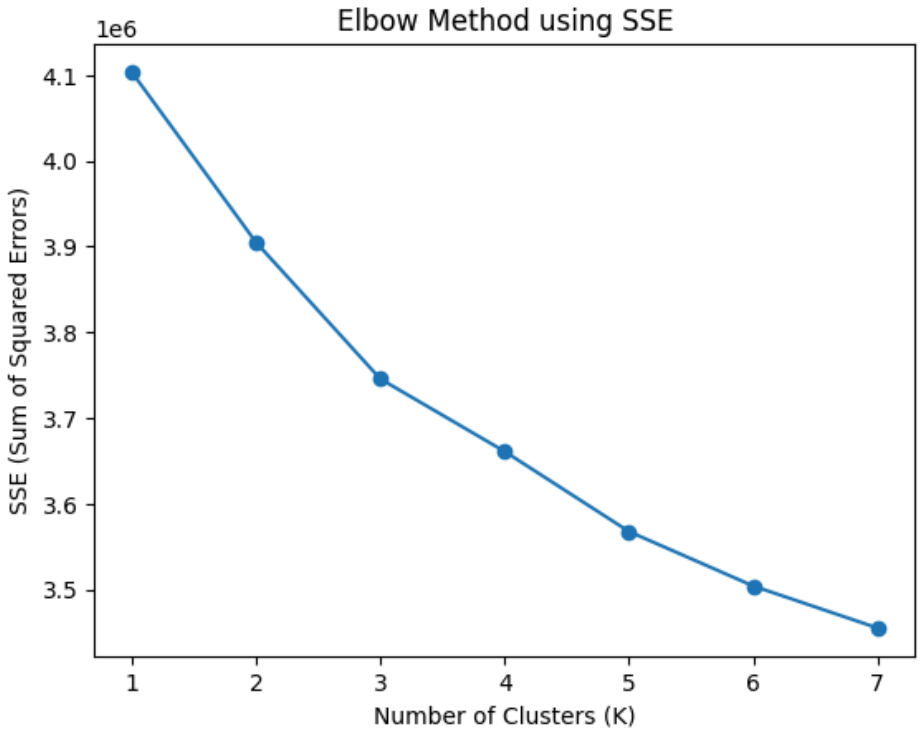
- **R^2 score and Loss:**

  As we did not use Linear Regression, it is non-applicable.

7. Model Training & Testing (Unsupervised)

- **Optimal Number of Clusters**

  The elbow method was used to inspect how inertia (within-cluster sum of squares) changes with the number of clusters.

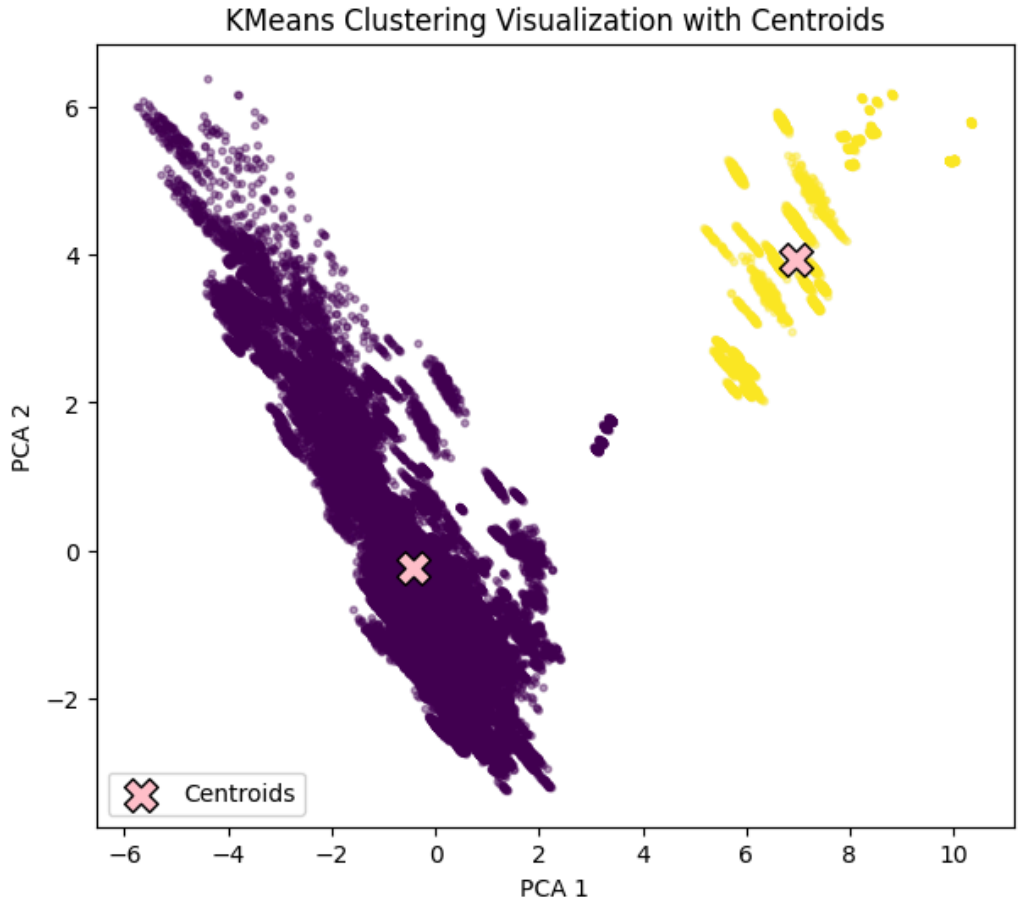


The inertia curve showed no sharply defined elbow, suggesting that the data does not contain clearly separated natural clusters. K = 2 was therefore selected based on the two known edibility classes, allowing the resulting clusters to be compared with the true class labels.

- **Feature Preparation**

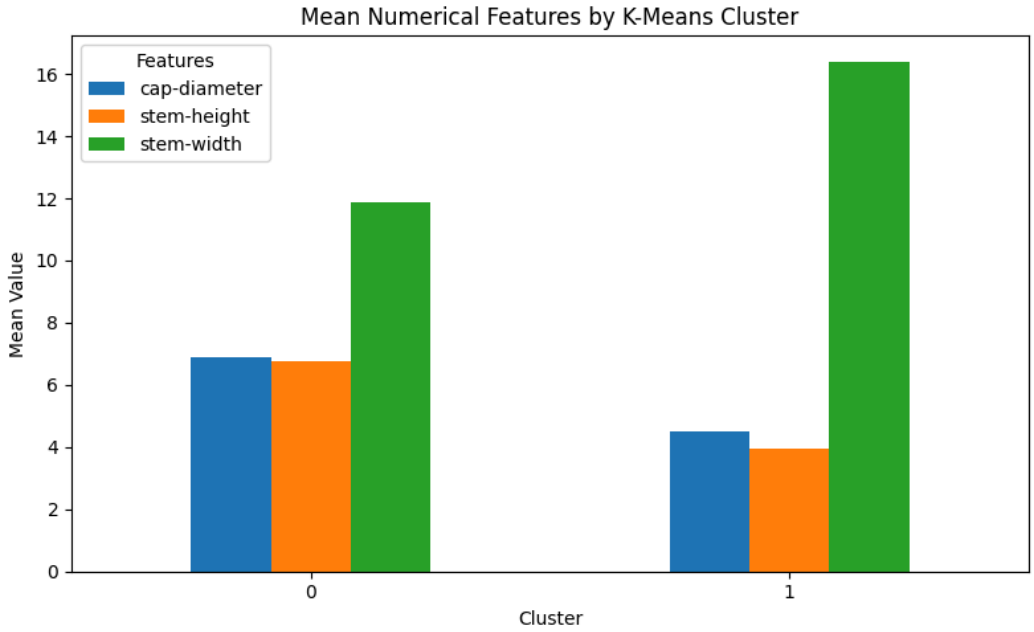
  The same standardized, one-hot encoded 84-feature matrix used for supervised models was reused for clustering, ensuring an identical feature representation.

- **Cluster Visualization and Centroid Analysis**



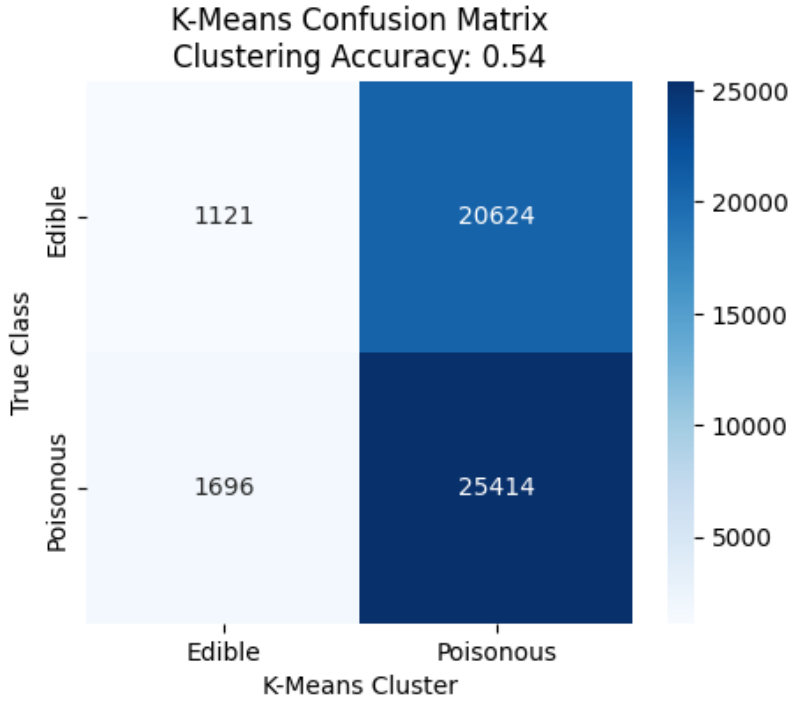
PCA was used to project the 84-dimensional feature space into two dimensions for visualization. The K-Means clusters appear clearly separated, with their centroids marking the center of each group. However, comparison with the true class labels shows that both edible and poisonous mushrooms occur within each cluster, indicating that the clusters do not directly correspond to edibility classes.

- **Cluster Interpretation**
  - Numeric Feature Means by K-Means Cluster



Stem-width and cap-diameter were the most different features between the two clusters, meaning K-Means is primarily grouping mushrooms by overall size rather than by edibility-relevant traits.

  - Confusion Matrix



K-Means confusion matrix showing the correspondence between the generated clusters and the actual mushroom classes. The clustering results achieved an accuracy of 54% when the generated clusters were compared with the known mushroom class labels.

8. Conclusion

The results show that the supervised models were effective at classifying mushrooms as edible or poisonous, with the non-linear models performing better than Logistic Regression. This indicates that mushroom edibility depends on combinations of features rather than a simple linear relationship. The correlation analysis also supports this, as no individual feature showed a strong correlation with the target.

K-Means, using two clusters, achieved approximately 54.3% agreement with the actual classes. This relatively low performance suggests that the two edibility classes are not naturally separated based only on overall similarity in the feature space. The cluster analysis also showed differences in numeric features such as stem-width and cap-diameter, suggesting that K-Means was grouping mushrooms based more on general characteristics than edibility.

The strong supervised-model performance can be attributed to the models being able to learn combinations of categorical and numerical features that are difficult to capture using simple linear relationships. The dataset also contains a large number of encoded categorical features, which provides the models with detailed information for distinguishing between the two classes.

The main challenges faced were handling missing values, deciding which highly incomplete features to remove, encoding the categorical variables, and managing the high-dimensional feature space created by one-hot encoding. Five features with more than 50% missing values were removed, while the remaining missing values were handled using a separate “missing” category.